

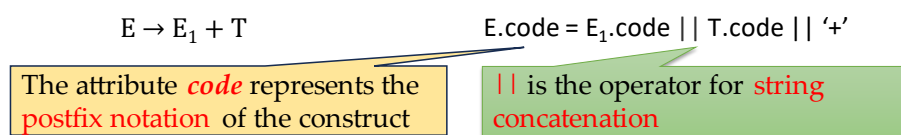
A Brief Introduction

- **Syntax-directed translation (语法制导的翻译)** is the process of language translation guided by context-free grammars
 - Here, “language translation” is in a broad sense
 - Transforming infix expressions (中缀表达式) to postfix expressions (后缀表达式)
 - A language construct is typically made of smaller constructs
 - The semantics of a construct can be **synthesized** from its constituent constructs’ semantics
 - The type of the expression $x + y$ is determined by the type of x and y , and the operator $+$
 - Or **inherited** from other constructs (e.g., siblings in the parse tree)
 - In “`int x`”, the type of x is determined by the type specifier to the left of x

- 读取源程序的输入字符，将其归类为词素，并生成词素序列
- 必要时将词目添加到符号表中
- 删除注释和空白（空格、换行符、制表符等）
- 将错误信息与行号关联起来（跟踪换行符）

Syntax-Directed Definitions

- A **syntax-directed definition** (**语法制导定义, SDD**) is a context-free grammar together with **attributes** and **rules**
 - A set of **attributes** (**属性**) is associated with each grammar symbol*
 - Can be anything, e.g., data type of expressions, # instructions in the generated code
 - A **semantic rule** (**语义规则**) is associated with a production and describes how attributes are computed



*Grammar symbols represent language constructs. Nonterminal nodes and subtrees rooted at these nodes correspond to productions.

语法制导定义（SDD）是一种无上下文语法，同时包含属性和规则。

一组属性（attributes）与每个语法符号相关联*。

可以是任何属性，例如表达式的数据类型，# 生成代码中的指令

语义规则（semantic rule）与语法生成相关联，描述如何计算属性

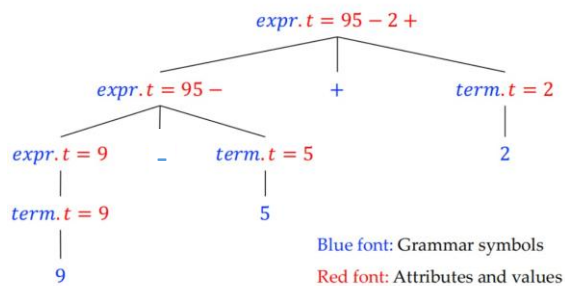
属性 code 表示结构体的后缀符号

|| 是字符串连接操作符

Annotated Parse Tree

- An **annotated parse tree** for **infix expression** 9 -5+2
- The attribute **t** represents postfix notation

$\text{expr} \rightarrow \text{expr} + \text{term}$
 $\text{expr} \rightarrow \text{expr} - \text{term}$
 $\text{expr} \rightarrow \text{term}$
 $\text{term} \rightarrow 0$
...
 $\text{term} \rightarrow 9$



注释解析树

下位表达式 9-5+2 的注释解析树

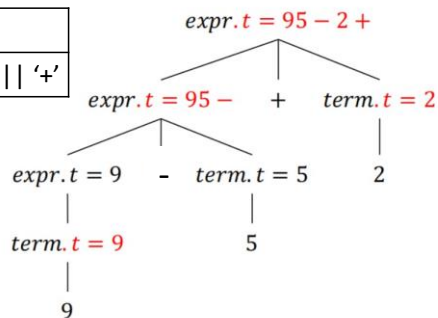
属性 t 代表后缀符号

Synthesized Attributes (综合属性)

- An attribute is said to be *synthesized* if its value at a parse-tree node N is only determined from attribute values at the children of N and at N itself (defined by a semantic rule associated with the production at N)

ProProduction	$\text{Expr} \rightarrow \text{expr}_1 + \text{term}$
Rule	$\text{Expr.t} = \text{expr}_1.\text{t} \mid \mid \text{term.t} \mid \mid '+'$

ProProduction	Term $\rightarrow$ 9
Rule	Term.t = '9'



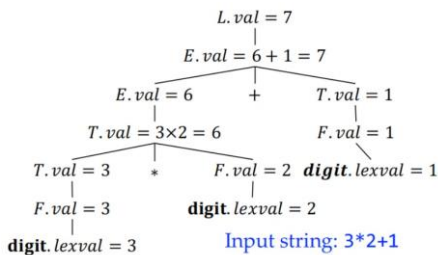
Terminals can also have synthesized attributes (lexical values), but there are not rules for computing the value of an attribute for a terminal.

如果一个属性在解析树节点 N 上的值仅由 N 的子节点和 N 本身的属性值决定, 则称该属性为合成属性;

A Complete Example of SDD

- The SDD below helps compute the value of an expression
- SDD's do not specify the evaluation order of attributes on a parse tree
 - Any order that computes an attribute after all other attributes that depends on is fine
 - Synthesized attributes have a nice property that they can be evaluated during a single bottom-up traversal of a parse tree (it is often unnecessary to explicitly create the tree)

PRODUCTION	SEMANTIC RULES
1) $L \rightarrow E \mathbf{n}$	$L.val = E.val$
2) $E \rightarrow E_1 + T$	$E.val = E_1.val + T.val$
3) $E \rightarrow T$	$E.val = T.val$
4) $T \rightarrow T_1 * F$	$T.val = T_1.val \times F.val$
5) $T \rightarrow F$	$T.val = F.val$
6) $F \rightarrow (E)$	$F.val = E.val$
7) $F \rightarrow \mathbf{digit}$	$F.val = \mathbf{digit.lexval}$



Inherited Attributes (继承属性)

- **Inherited attributes** have their value at a parse-tree node determined from attribute values at **the node itself, its parent, and its siblings** in the parse tree

We already have synthesized attributes. When are inherited attributes useful???

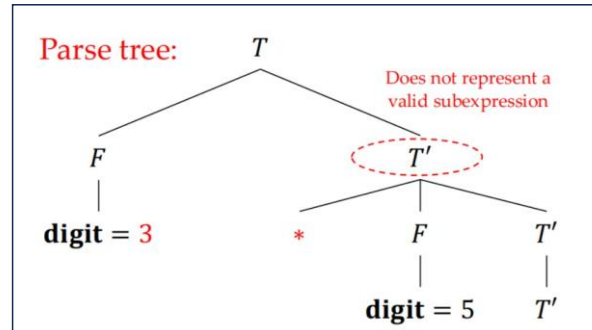
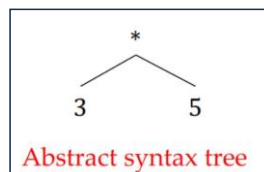


继承属性在解析树节点上的值由该节点本身、其父节点和解析树中同级节点的属性值决定

回答：继承属性在处理与上下文相关的问题时仍然是非常有用的。它为我们提供了一种在语法树中考虑外部信息的方法，从而使得我们可以更为精确和灵活地描述语法制导定义的属性

Top-Down Parse of 3*5

$T \rightarrow FT'$
 $T' \rightarrow *FT'$
 $T' \rightarrow \epsilon$
 $F \rightarrow \text{digit}$



For some grammars, the structure of the parse tree does not **match** the abstract syntax of the code (3 and * are in different subtrees)

Not all non-terminals in a parse tree correspond to language constructs, e.g., T' above.

对于某些语法，解析树的结构与代码的抽象语法不一致（3 和 * 位于不同的子树中）

SDD with Inherited Attributes

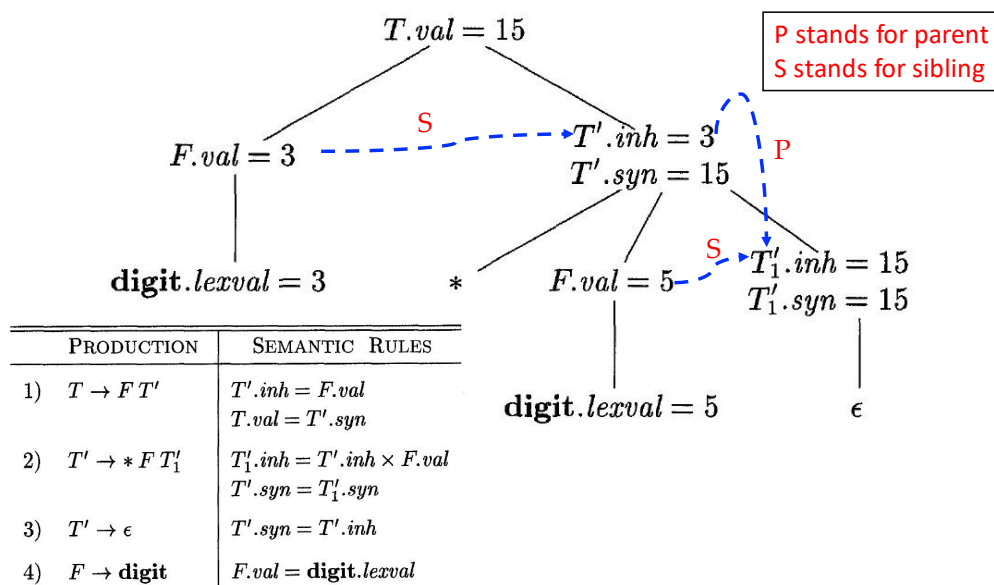
PRODUCTION	SEMANTIC RULES
1) $T \rightarrow F T'$	$T'.inh = F.val$ $T.val = T'.syn$
2) $T' \rightarrow * F T'_1$	$T'_1.inh = T'.inh \times F.val$ $T'.syn = T'_1.syn$
3) $T' \rightarrow \epsilon$	$T'.syn = T'.inh$
4) $F \rightarrow \text{digit}$	$F.val = \text{digit.lexval}$

The left operand of the operator * is inherited by T' and kept for later computation when T' further gets replaced

The inherited attribute of T' is not defined by a rule associated with the production (2) or (3), whose head is T'

运算符 $*$ 的左操作数由 T' 继承, 并在 T' 进一步被替换时保留用于以后的计算。

Annotated Parse Tree for $3*5$



先从具体的输入开始（这是语义计算的起点）结合分析树的结构来分析

根据 4) 产生式及其语义规则, 得到 $F.val$,

根据 2) 产生式及其语义规则, 要想计算 $T'_1.inh = T'.inh$ (未知) $\times F.val$ (5)。要

计算 $T'.syn = T'_1.syn$ (未知)

根据 1) 产生式及其语义规则, $T'.inh = F.val$ (已知) $= 3$, $T.val = T'.syn$ (未知)

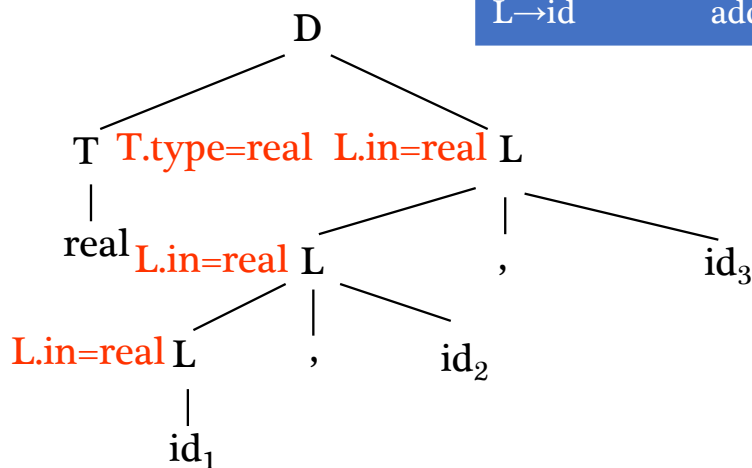
返回 2) 产生式及其语义规则, 计算 $T'_1.inh = T'.inh$ (3) $\times F.val$ (5) $= 15$ 。

根据 3) 产生式及其语义规则, 计算 $T'_1.syn = T'_1.inh = 15$ 。

返回 2) 产生式及其语义规则, 计算 $T'.syn = T'_1.syn$ (15)

返回 1) 产生式及其语义规则, 计算 $T.val = T'.syn$ (15)

Annotated Parse Tree
for real id_1, id_2, id_3



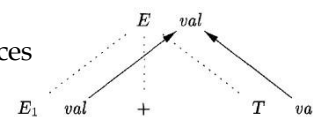
PRODUCTIONS	SEMANTIC RULES
$D \rightarrow TL$	$L.in := T.type$
$T \rightarrow int$	$T.type := integer$
$T \rightarrow real$	$T.type := real$
$L \rightarrow L_1, id$	$L_1.in := L.in$ $addtype(id.entry, L.in)$
$L \rightarrow id$	$addtype(id.entry, L.in)$

name	type
...	...
id_3	real
id_2	real
id_1	real

Symbol table

Evaluation Orders for SDD's

- Given parse tree nodes $N, M_1, M_2, M_3, \dots, M_k$, if the attribute a of N is defined as $N.a = f(M_1.a_1, M_2.a_2, \dots, M_k.a_k)$ then in order to compute $N.a$, we must first compute $M_i.a_i$ ($1 \leq i \leq k$)
- Dependency graphs* (依赖图) are a useful tool for determining evaluation orders
 - Depict the **information flow** among the attribute instances in **a particular parse tree**
 - Model the **partial order** among attribute instances



给定解析树节点 $N, M_1, M_2, M_3, \dots, M_k$, 如果 N 的属性 a 定义为 $N.a = f(M_1.a_1, M_2.a_2, \dots, M_k.a_k)$, 那么为了计算 $N.a$, 我们必须先计算 M_1 。

依赖图 (依赖图) 是确定评估顺序的有用工具

描述特定解析树中属性实例之间的信息流

为属性实例之间的部分顺序建模

Dependency Graph

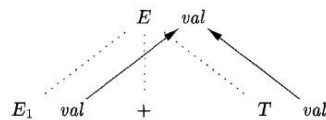
- An **edge** from one **attribute instance** (a_1) to another (a_2) means that the value of a_1 is needed to compute the value of a_2
- If there is any **cycle** in a dependency graph, we cannot find an order to compute the value of all attribute instances

PRODUCTION

$E \rightarrow E_1 + T$

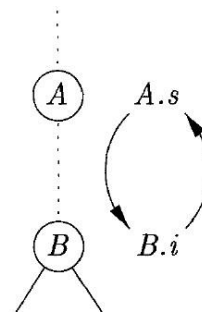
SEMANTIC RULE

$E.val = E_1.val + T.val$



Dotted lines: parse tree edges

Solid lines: dependency graph edges



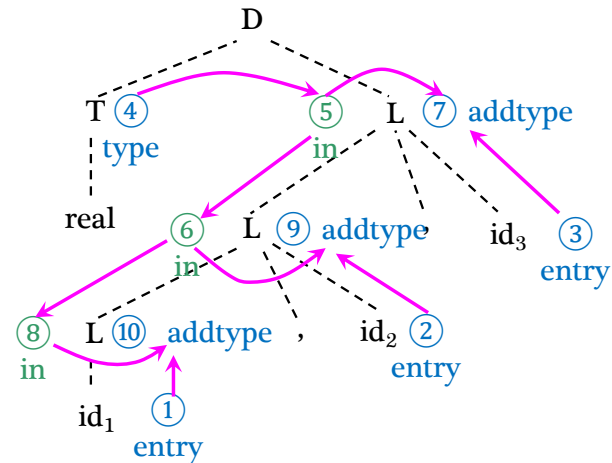
从一个属性实例 (a_1) 到另一个属性实例 (a_2) 的边表示计算 a_2 的值需要 a_1 的值。

如果依赖关系图中存在循环, 我们就无法找到计算所有属性实例值的顺序。

Example1: Parsing real id₁, id₂, id₃

Attribute values can be computed according to any *topological sort* (拓扑排序) of the graph, e.g., 1, 2, 3, ..., 10 in the example below

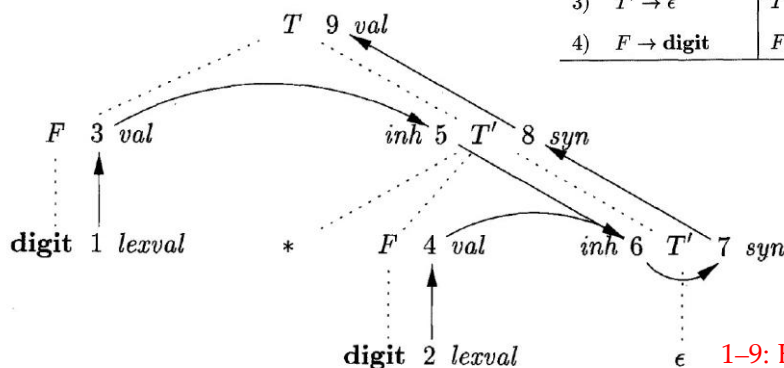
PRODUCTIONS	SEMANTIC RULES
$D \rightarrow TL$	$L.in := T.type$
$T \rightarrow int$	$T.type := integer$
$T \rightarrow real$	$T.type := real$
$L \rightarrow L_1, id$	$L_1.in := L.in$ $addtype(id.entry, L.in)$
$L \rightarrow id$	$addtype(id.entry, L.in)$



1-10: Evaluation order

Example2: Parsing 3*2

PRODUCTION	SEMANTIC RULES
1) $T \rightarrow F T'$	$T'.inh = F.val$ $T.val = T'.syn$
2) $T' \rightarrow * F T'_1$	$T'_1.inh = T'.inh \times F.val$ $T'.syn = T'_1.syn$
3) $T' \rightarrow \epsilon$	$T'.syn = T'.inh$
4) $F \rightarrow digit$	$F.val = digit.lexval$



1-9: Evaluation order

属性值可根据图的任意拓扑排序 (拓扑排序) 计算, 例如下例中的 1、2、3、.....、9

Ordering the Evaluation of Attributes

- Given an SDD, it is very hard to tell whether there exist any parse trees whose dependency graphs have cycles
- In practice, translations can be implemented using classes of SDD's that guarantee an evaluation order*

S-attributed SDD's

L-attributed SDD's

*The dependency graphs for such SDDs are directed acyclic graphs

如果一个属性文法不存在属性之间的循环依赖关系，则称该文法为良定义的
一个依赖图的任何拓扑排序都给出一个语法树中结点的语义规则计算的有效顺序
在给定 SDD 的情况下，很难判断是否存在依赖图具有循环的解析树。

在实践中，可以使用保证求值顺序的 SDD 类来实现翻译*。(这种 SDD 的依赖图是有向无环图)

S 属性 SDD

L 属性 SDD

有向无环图 directed acyclic graphs

S-Attributed Definitions

- An SDD is **S-attributed** if every attribute is synthesized

PRODUCTION	SEMANTIC RULES
1) $L \rightarrow E \mathbf{n}$	$L.val = E.val$
2) $E \rightarrow E_1 + T$	$E.val = E_1.val + T.val$
3) $E \rightarrow T$	$E.val = T.val$
4) $T \rightarrow T_1 * F$	$T.val = T_1.val \times F.val$
5) $T \rightarrow F$	$T.val = F.val$
6) $F \rightarrow (E)$	$F.val = E.val$
7) $F \rightarrow \mathbf{digit}$	$F.val = \mathbf{digit.lexval}$

Intuitively, there cannot be cycles in the dependency graph of any parse tree, since edges always go from children nodes to parent nodes, never the other way around.

S 属性定义

如果每个属性都是合成的，那么语法制导定义就是 S 属性的

直观地说，任何解析树的依赖关系图中都不可能存在循环，因为边总是从子节点到父节点，而不是相反。

S-Attributed Definitions Cont.

- When an SDD is S-attributed, we can evaluate its attributes in any **bottom up order** of the parse-tree nodes
e.g., **postorder traversal (后序遍历)** of the parse tree
- S-attributed definitions can be easily implemented during **bottom-up parsing** (corresponds to a postorder traversal)

```
postorder(N) {  
    for ( each child C of N, from the left ) postorder(C);  
    evaluate the attributes associated with node N;  
}
```

当 SDD 具有 S 属性时，我们可以按照解析树节点的任何自下而上的顺序评估其属性。

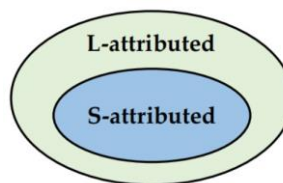
例如，解析树的后序遍历 (postorder traversal

S 属性定义可以在自下而上的解析过程中轻松实现 (相当于后序遍历)

L-Attributed Definitions

- An SDD is *L-attributed* if for each production $A \rightarrow X_1X_2...X_n$, for each $J = 1 \dots n$, each inherited attribute of x_j depends on only:
 - the attributes of $x_1, \dots, x_{j-1}$ (either synthesized or inherited), or
 - the inherited attributes of A

Dependency-graph edges can go from left to right (for a parse tree), but not right to left (hence "L-attributed")



L 属性定义

如果对于每个产品 $A \rightarrow X_1X_2...X_n$ ，对于每个 $J = 1 \dots n$ ， x_j 的每个继承属性仅取决于以下属性，则 SDD 是 L 属性 SDD：

$x_1, \dots, x_{j-1}$ 的属性（合成的或继承的），

或 A 的继承属性

依赖关系图的边可以从左到右（对于解析树而言），但不能从右到左（因此称为 "L-属性"）。

L-Attributed Definition Example

PRODUCTION	SEMANTIC RULES
1) $T \rightarrow F T'$	$T'.inh = F.val$ Left sibling's attribute $T.val = T'.syn$
2) $T' \rightarrow * F T'_1$	$T'_1.inh = T'.inh \times F.val$ $T'.syn = T'_1.syn$ • Parent's inherited attribute • Left sibling's attribute
3) $T' \rightarrow \epsilon$	$T'.syn = T'.inh$
4) $F \rightarrow \text{digit}$	$F.val = \text{digit.lexval}$

Synthesized attributes: val, syn, lexval . Inherited attributes: inh

- 左同胞的属性
- 父辈的继承属性 - 左同胞的属性

Attribute Evaluation for L-Attributed Definitions

Input: A node n in a parse tree T

Output: An evaluation order for the attributes of the subtree rooted at n

Procedure depth_first(n)*
Begin

When evaluating the inherited attributes of a node, the attributes of nodes to its left have been evaluated

↑
Guarantee

for every child m of n from left to right do begin
 evaluate the inherited attributes of m ;
 depth_first(m); // here m 's synthesized attributes will be evaluated
 end
 evaluate the synthesized attributes of n ;

end

* The inherited attributes of n are computed before calling depth_first(n), as indicated by the for body

在计算节点的继承属性时，已计算了其左侧节点的属性

前序遍历计算继承属性

输入: 解析树 T 中的节点 n

输出: 以 n 为根的子树属性的评估顺序

程序 $\text{depth_first}(n)^*$

开始

对于 n 的每个子节点 m ，从左到右依次执行 开始

 评估 m 的继承属性;

$\text{depth_first}(m)$; // 这里将评估 m 的合成属性

结束

评估 n 的合成属性;

保证: 在评估节点的继承属性时，已评估了其左侧节点的属性

end

Attribute Evaluation for L-Attributed Definitions

Input: A node n in a parse tree T

Output: An evaluation order for the attributes of the subtree rooted at n

Procedure $\text{depth_first}(n)^*$
Begin

When evaluating the inherited attributes of a node, the inherited attributes of its parent node have been evaluated

for every child m of n from left to right do begin

 evaluate the inherited attributes of m ;

$\text{depth_first}(m)$; // here m 's synthesized attributes will be evaluated

end

 evaluate the synthesized attributes of n ;

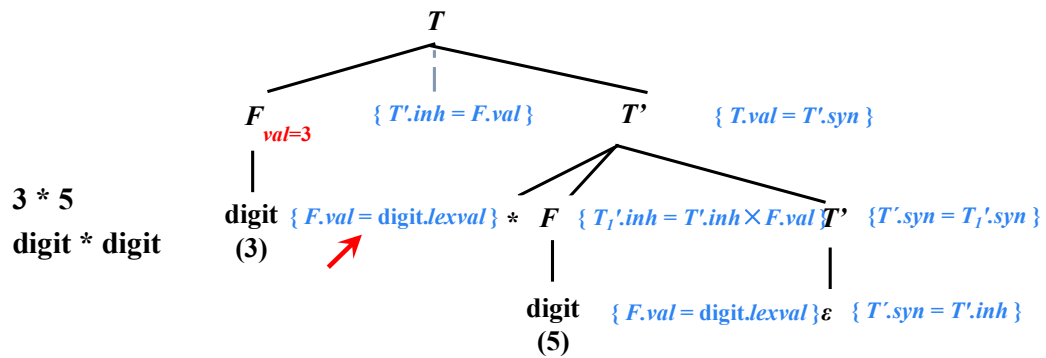
end

Guarantee

* The inherited attributes of n are computed before calling $\text{depth_first}(n)$, as indicated by the for body

保证: 在计算节点的继承属性时，已计算出其父节点的继承属性

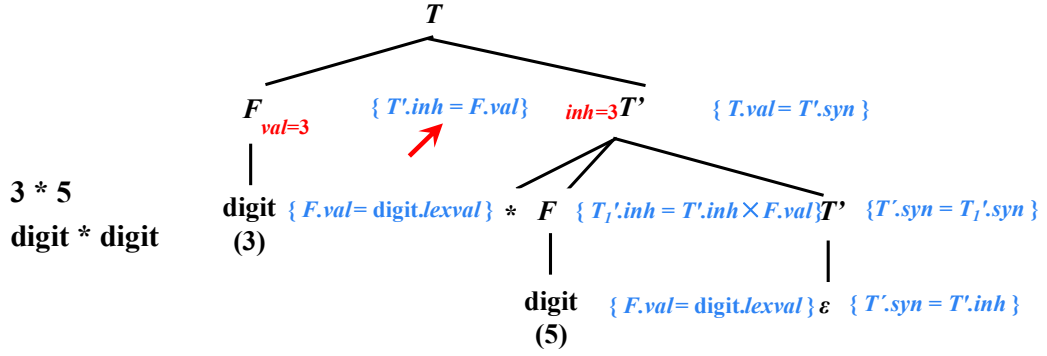
PRODUCTION	SEMANTIC RULES
1) $T \rightarrow F T'$	$T'.inh = F.val$ $T.val = T'.syn$
2) $T' \rightarrow * F T'_1$	$T'_1.inh = T'.inh \times F.val$ $T'.syn = T'_1.syn$
3) $T' \rightarrow \epsilon$	$T'.syn = T'.inh$
4) $F \rightarrow \text{digit}$	$F.val = \text{digit.lexval}$



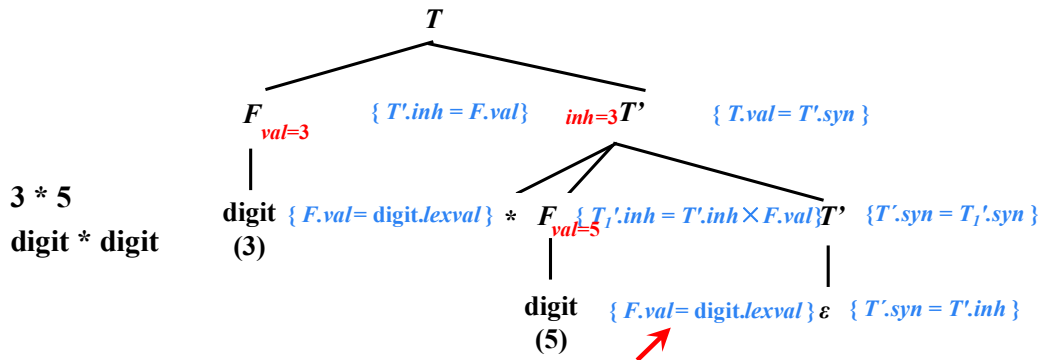
➤ 通用方法

- 首先建立一棵语法分析树，然后按照从左到右的深度优先顺序来执行这些动作，也就是说在一个前序遍历过程中执行

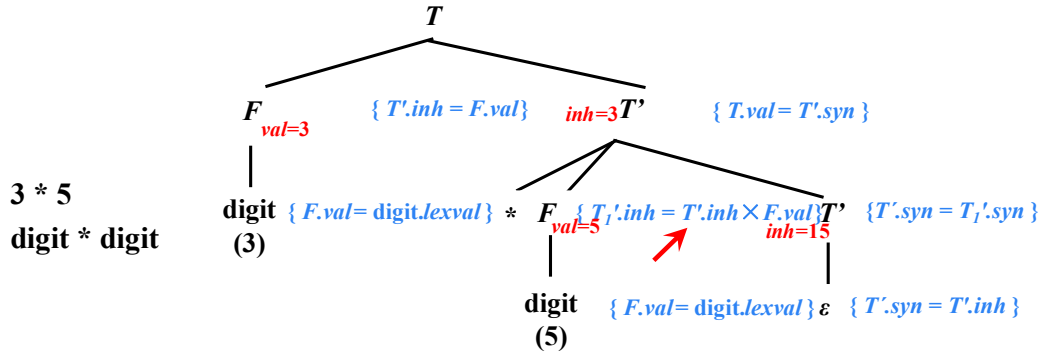
PRODUCTION	SEMANTIC RULES
1) $T \rightarrow F T'$	$T'.inh = F.val$ $T.val = T'.syn$
2) $T' \rightarrow * F T'_1$	$T'_1.inh = T'.inh \times F.val$ $T'.syn = T'_1.syn$
3) $T' \rightarrow \epsilon$	$T'.syn = T'.inh$
4) $F \rightarrow \text{digit}$	$F.val = \text{digit.lexval}$



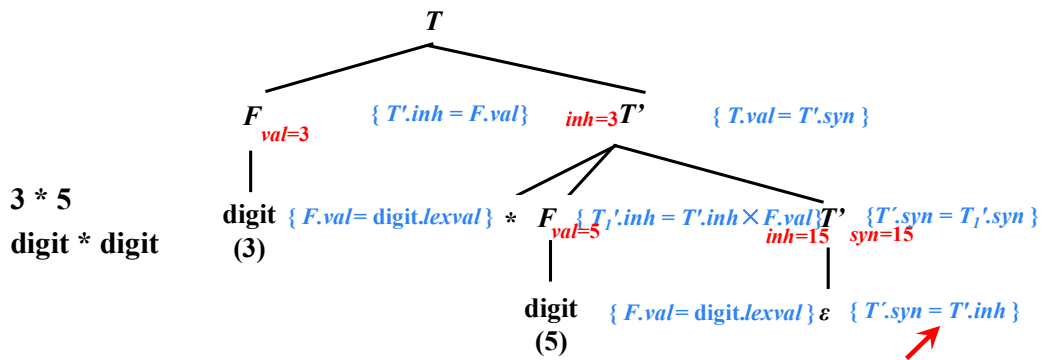
PRODUCTION	SEMANTIC RULES
1) $T \rightarrow F T'$	$T'.inh = F.val$ $T.val = T'.syn$
2) $T' \rightarrow * F T'_1$	$T'_1.inh = T'.inh \times F.val$ $T'.syn = T'_1.syn$
3) $T' \rightarrow \epsilon$	$T'.syn = T'.inh$
4) $F \rightarrow \text{digit}$	$F.val = \text{digit.lexval}$



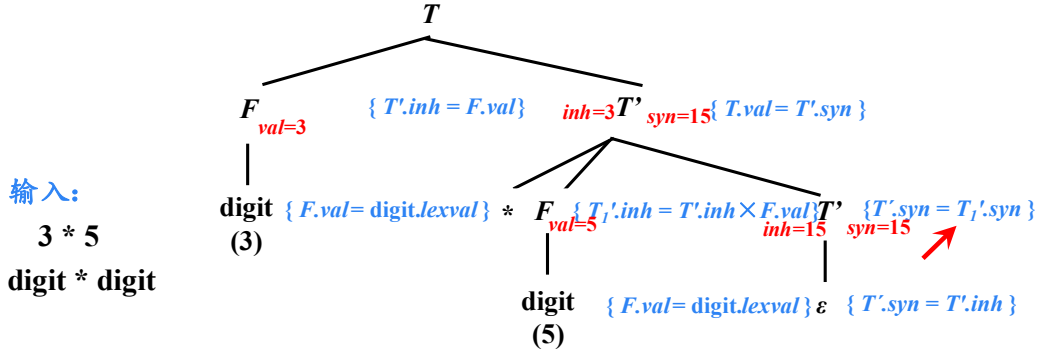
PRODUCTION	SEMANTIC RULES
1) $T \rightarrow F T'$	$T'.inh = F.val$ $T.val = T'.syn$
2) $T' \rightarrow * F T'_1$	$T'_1.inh = T'.inh \times F.val$ $T'.syn = T'_1.syn$
3) $T' \rightarrow \epsilon$	$T'.syn = T'.inh$
4) $F \rightarrow \text{digit}$	$F.val = \text{digit.lexval}$



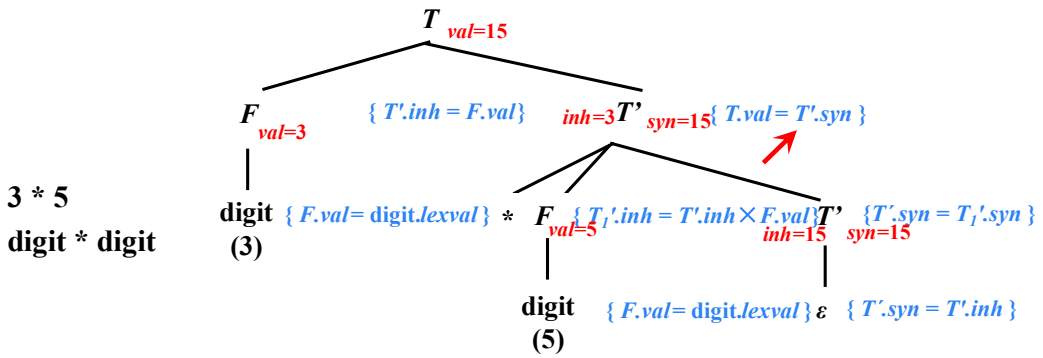
PRODUCTION	SEMANTIC RULES
1) $T \rightarrow F T'$	$T'.inh = F.val$ $T.val = T'.syn$
2) $T' \rightarrow * F T'_1$	$T'_1.inh = T'.inh \times F.val$ $T'.syn = T'_1.syn$
3) $T' \rightarrow \epsilon$	$T'.syn = T'.inh$
4) $F \rightarrow \text{digit}$	$F.val = \text{digit.lexval}$



PRODUCTION	SEMANTIC RULES
1) $T \rightarrow F T'$	$T'.inh = F.val$ $T.val = T'.syn$
2) $T' \rightarrow * F T'_1$	$T'_1.inh = T'.inh \times F.val$ $T'.syn = T'_1.syn$
3) $T' \rightarrow \epsilon$	$T'.syn = T'.inh$
4) $F \rightarrow \text{digit}$	$F.val = \text{digit.lexval}$



PRODUCTION	SEMANTIC RULES
1) $T \rightarrow F T'$	$T'.inh = F.val$ $T.val = T'.syn$
2) $T' \rightarrow * F T'_1$	$T'_1.inh = T'.inh \times F.val$ $T'.syn = T'_1.syn$
3) $T' \rightarrow \epsilon$	$T'.syn = T'.inh$
4) $F \rightarrow \text{digit}$	$F.val = \text{digit.lexval}$



Attribute Evaluation for L-Attributed Definitions

Input: A node n in a parse tree T

Output: An evaluation order for the attributes of the subtree rooted at n

Procedure depth_first(n)*

Begin

for every child m of n from left to right **do begin**

evaluate the inherited attributes of m ;

depth_first(m); // here m 's synthesized attributes will be evaluated

end

evaluate the synthesized attributes of n ;

end

Can be implemented in
top-down parsing

* The inherited attributes of n are computed before calling depth_first(n), as indicated by the for body

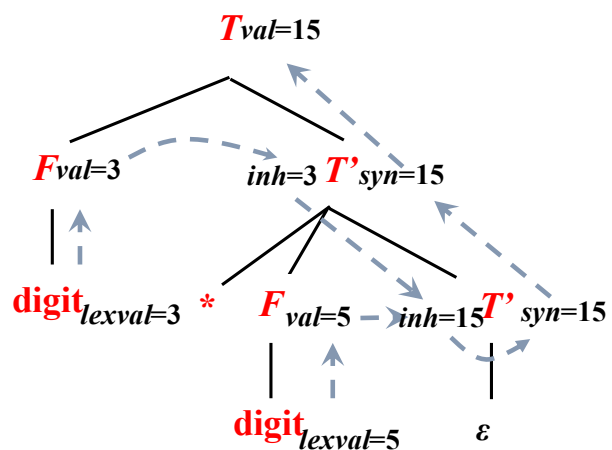
可自顶向下解析

Can be implemented in
top-down parsing

input:

3 * 5

digit * digit

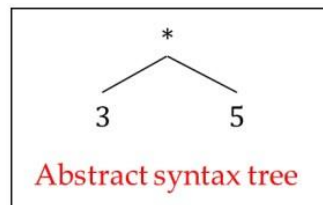


通用方法（两趟）

首先建立一棵语法分析树，然后按照从左到右的深度优先顺序来执行这些动作，也就是说在一个前序遍历过程中执行(继承属性的计算)，然后再执行一个后序遍历完成综合属性的计算。

Construction of Syntax Tree (1)

- **Abstract syntax tree** (revisited)
 - Each interior node N represents a **construct** (corresponding to an **operator**)
 - The children of N represent the meaningful **components of the construct** represented by N (corresponding to **operands**)



抽象语法树（重温）

每个内部节点 N 代表一个结构体（对应一个运算符）

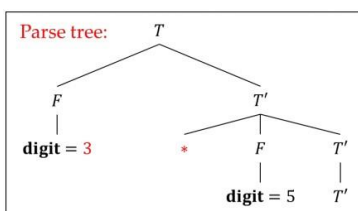
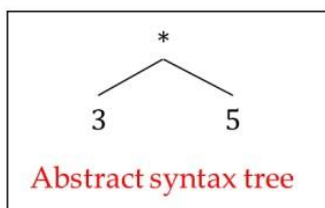
N 的子节点代表 N 所代表结构的有意义成分（相当于操作数）

Construction of Syntax Tree (2)

- Syntax tree vs. parse tree

In a syntax tree, interior nodes represent **programming constructs**, while in a parse tree, interior nodes represent **nonterminals***

A parse tree is also called a **concrete syntax tree**, and the underlying grammar is called a **concrete syntax** for the language



* Not all nonterminals represent programming constructs, e.g., those introduced to eliminate left recursions (in the earlier L-attributed definition example)

语法树与解析树

在语法树中，内部节点代表编程结构，而在解析树中，内部节点代表非终结符*。

解析树也称为具体语法树，其底层语法称为语言的具体语法

Construction of Syntax Tree (3)

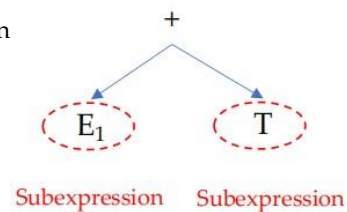
- An S-attributed SDD for building syntax trees for simple expressions

Each node of the syntax tree is implemented as an **object** with a field *op*, representing the label of the node, and some additional fields

Leaf node: one additional field holding the lexical value

Interior node: # additional fields = # of children

PRODUCTION	SEMANTIC RULES
1) $E \rightarrow E_1 + T$	$E.node = \text{new Node}('+', E_1.node, T.node)$
2) $E \rightarrow E_1 - T$	$E.node = \text{new Node}('-', E_1.node, T.node)$
3) $E \rightarrow T$	$E.node = T.node$
4) $T \rightarrow (E)$	$T.node = E.node$
5) $T \rightarrow \text{id}$	$T.node = \text{new Leaf}(\text{id}, \text{id.entry})$
6) $T \rightarrow \text{num}$	$T.node = \text{new Leaf}(\text{num}, \text{num.val})$



Fall 2024

BUPT Compilers

38

为简单表达式构建语法树的 S 属性 SDD

语法树的每个节点都是一个对象，其中有一个字段 *op*（代表节点的标签）和一些附加字段

叶节点：一个包含词法值的附加字段

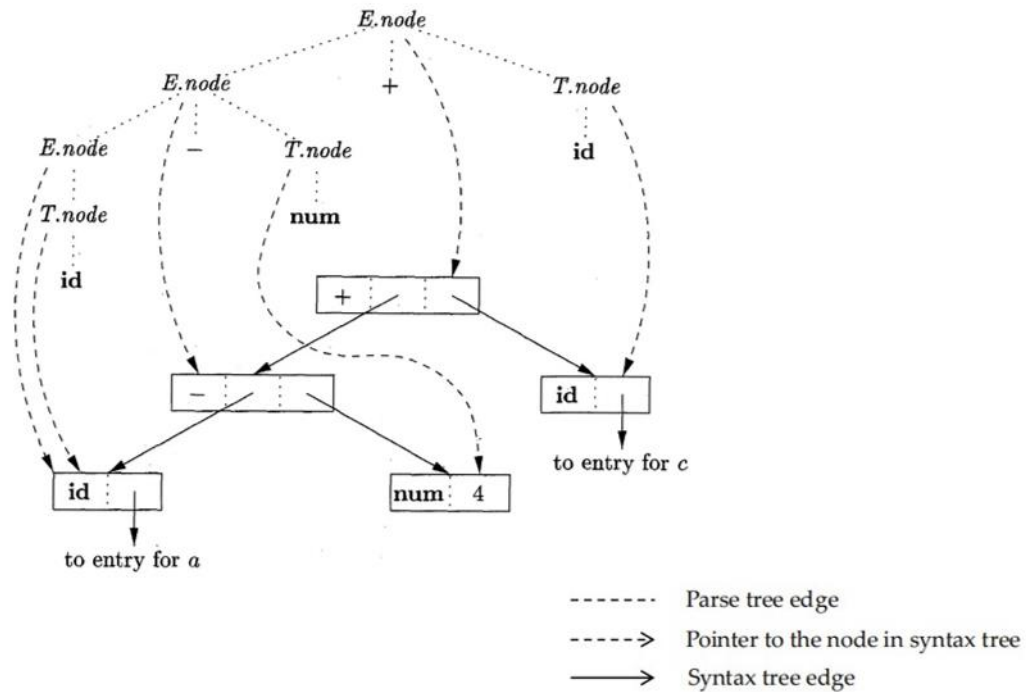
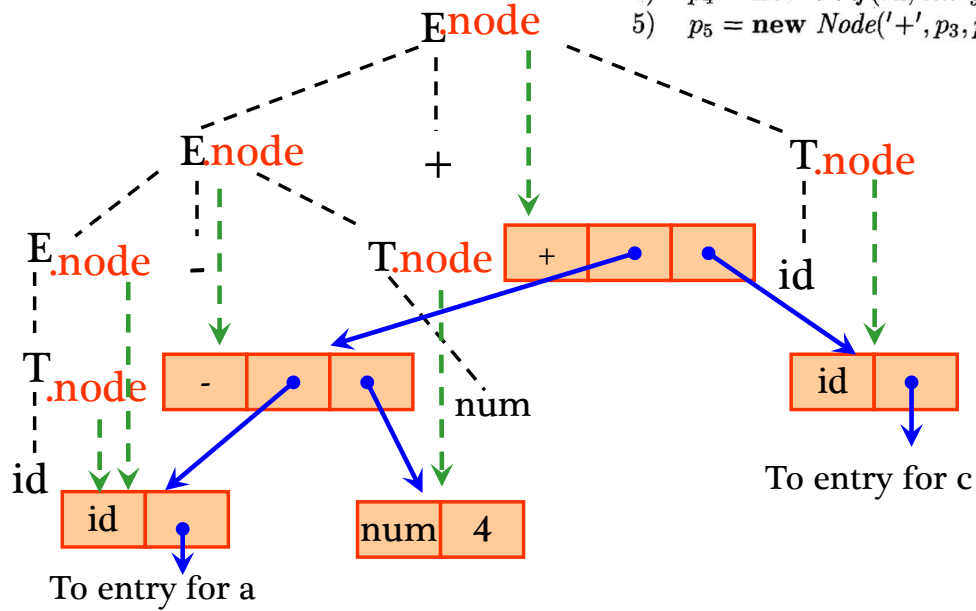
内部节点：额外的字段数量等于子节点的数量

Construction of Syntax Tree (4)

Input expression: $a - 4 + c$

Steps (bottom-up evaluation):

- 1) $p_1 = \text{new Leaf}(\text{id}, \text{entry-a});$
- 2) $p_2 = \text{new Leaf}(\text{num}, 4);$
- 3) $p_3 = \text{new Node}('-', p_1, p_2);$
- 4) $p_4 = \text{new Leaf}(\text{id}, \text{entry-c});$
- 5) $p_5 = \text{new Node}('+', p_3, p_4);$



Computing the Structure of a Type

```
int[2][3] a = ...;
```

What is the type of **a**?



- Leave it to the next chapter...

计算类型的结构

Syntax-Directed Translation Schemes

- SDD's tell us what to do (**high-level specifications**) in the translation, but not how to do
- **Syntax-directed translation schemes** (SDT's, 语法制导的翻译方案) specify more details on how to do the translation
- An SDT is a context-free grammar with semantic actions (program fragments) embedded within production bodies
 - Differ from the semantic rules in SDD's
 - Semantic actions can appear **anywhere** within a production body

SDD 告诉我们在翻译中要做什么（高级规范），但不告诉我们如何做

语法制导的翻译方案（SDT's, Syntax-directed translation schemes）规定了如何进行翻译的更多细节。

SDT 是一种无上下文语法，其语义动作（程序片段）嵌入在生产体中：

与 SDD 的语义规则不同

语义动作可以出现在制作体的任何地方

Implementation of SDT's

- Any SDT can be implemented by:
 - First building a parse tree
 - Treating semantic actions as “virtual” parse-tree nodes
 - Performing a left-to-right depth-first (i.e., preorder) tree traversal and triggering the actions when visiting the corresponding nodes

任何 SDT 都可以通过以下方式实现：

首先构建一棵解析树

将语义操作视为 “虚拟” 解析树节点

执行从左到右的深度优先（即预排序）树遍历，并在访问相应节点时触发操作

An Example SDT (1)

- The SDT below implements a simple calculator

SDT	$L \rightarrow E \text{ n}$	$\{ \text{print}(E.val); \}$	Semantic actions: Real code in {}
	$E \rightarrow E_1 + T$	$\{ E.val = E_1.val + T.val; \}$	
	$E \rightarrow T$	$\{ E.val = T.val; \}$	
	$T \rightarrow T_1 * F$	$\{ T.val = T_1.val \times F.val; \}$	
	$T \rightarrow F$	$\{ T.val = F.val; \}$	
	$F \rightarrow (E)$	$\{ F.val = E.val; \}$	
	$F \rightarrow \text{digit}$	$\{ F.val = \text{digit.lexval}; \}$	
SDD	PRODUCTION	SEMANTIC RULES	Semantic rules: Definitions (not code)
	1) $L \rightarrow E \text{ n}$	$L.val = E.val$	
	2) $E \rightarrow E_1 + T$	$E.val = E_1.val + T.val$	
	3) $E \rightarrow T$	$E.val = T.val$	
	4) $T \rightarrow T_1 * F$	$T.val = T_1.val \times F.val$	
	5) $T \rightarrow F$	$T.val = F.val$	
	6) $F \rightarrow (E)$	$F.val = E.val$	
	7) $F \rightarrow \text{digit}$	$F.val = \text{digit.lexval}$	

1. SDD (Syntax Directed Definition) :

- SDD 定义了与语法产生式关联的属性和规则。它为每个语法产生式指定一个或多个属性计算或语义动作。
- SDD 关注于如何为语法树中的节点计算属性值，但不一定涉及产生中间代码或目标代码的具体操作。
- 它可以有合成属性和继承属性，因此可以描述为 S 属性的（只有合成属性）或 L 属性的（允许继承属性但有限制）。

2. SDT (Syntax Directed Translation) :

- SDT 是 SDD 的一个特例，它专门用于生成中间代码或目标代码。换句话说，SDT 的目的是翻译，它定义了与语法产生式关联的动作，这些动作通常在解析时执行。
- 当解析器在处理产生式时，与该产生式关联的 SDT 动作被执行，从而生成代码或进行其他翻译工作。
- 动作可以嵌入到产生式中，例如，在 YACC 或 Bison 这样的工具中，可以直接在语法规则中插入 C 代码作为动作。

总结差异:

- SDD 更为通用, 关注于为语法树的节点计算属性。这些属性可以用于多种目的, 如类型检查、变量绑定等。
- SDT 特化于翻译任务, 即将源代码转换为中间代码或目标代码。它通常涉及在解析过程中执行特定的动作。

尽管有这些区别, 但在实际应用中, SDD 和 SDT 的界限可能会模糊, 因为编译器的语义分析和翻译阶段通常是交织在一起的

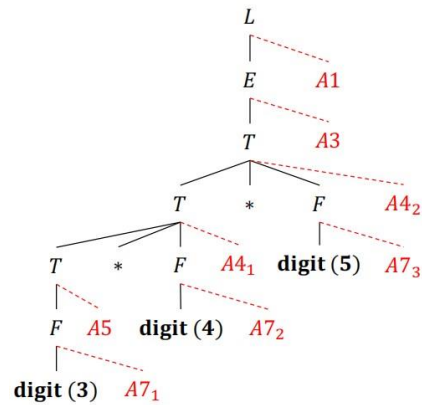
An Example SDT (2)

- Parse and calculate $3*4*5$

L	$\rightarrow$	$E \mathbf{n}$	$\{ \text{print}(E.val); \}$	$A1$
E	$\rightarrow$	$E_1 + T$	$\{ E.val = E_1.val + T.val; \}$	$A2$
E	$\rightarrow$	T	$\{ E.val = T.val; \}$	$A3$
T	$\rightarrow$	$T_1 * F$	$\{ T.val = T_1.val \times F.val; \}$	$A4$
T	$\rightarrow$	F	$\{ T.val = F.val; \}$	$A5$
F	$\rightarrow$	(E)	$\{ F.val = E.val; \}$	$A6$
F	$\rightarrow$	digit	$\{ F.val = \mathbf{digit.lexval}; \}$	$A7$

Order of actions:

$A7_1, A5, A7_2, A4_1, A7_3, A4_2, A3, A1$



SDT's With Actions Inside Productions

$$B \rightarrow X\{a\}Y$$

- The action a should be done after we have recognized X (if X is a terminal) or all the terminals derived from X (if X is a nonterminal)
 - If the parse is **bottom-up**, we perform the **action a** as soon as X appears on the top of the parsing stack
 - If the parse is **top-down**, we perform the **action a** before attempting to expand Y (if Y is a nonterminal) or check for Y on the input (if Y is a terminal)

结合语法分析树理解

动作 a 应在我们识别完 X （如果 X 是终结符）或由 X 派生的所有终结符（如果 X 是非终结符）后执行

如果解析是自下而上的，那么一旦 X 出现在解析栈顶端，我们就执行操作 a （此时，有关 X 的所有信息都已在栈中）

如果解析是自上而下的，我们在尝试扩展 Y （如果 Y 是非终结符）或检查输入中的 Y （如果 Y 是终结符）之前执行操作 a （此时有关 X 的所有信息已分析完毕）

SDT's Implementable During Parsing

- In practice, SDT's are often implemented during parsing
without building a parse tree
- **Not all** SDT's can be implemented during parsing *
- Even if the underlying grammar is parsable by a method (e.g., LL, LR), after introducing semantic actions, the method may become inapplicable
- **Determine if an SDT can be implemented during parsing**
 - Introduce distinct *marker nonterminals* to replace each embedded action; each marker M has only one production $M \rightarrow \epsilon$
 - If the grammar with marker nonterminals can be parsed by a given method, then the SDT can be implemented during parsing

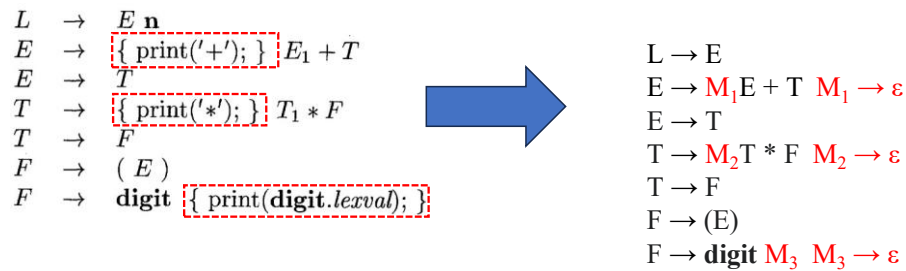
*Note that all SDT's can be implemented after building the parse tree, as earlier mentioned

注意，如前所述，所有 SDT 都可以在构建分析树后实施。

将语义动作标记非终结 M 来代替，对应的的产生式为 $M \rightarrow \epsilon$ ，得到文法可以被 LL 或 LR 分析器所分析，这样的 SDT 才能在语法分析时一并进行。

A Problematic SDT

- This SDT translates infix expression to prefix expressions



It is impossible to build parsing tables without conflicts using top-down or bottom-up parsing methods. This SDT cannot be implemented during parsing.

使用自上而下或自下而上的解析方法不可能建立没有冲突的解析表。

自下而上的解析方法来建立没有冲突的解析表。这种 SDT 无法在解析过程中执行。

Uses of SDT's

- We can use SDT's to implement two important classes of SDD's:

The underlying grammar is LR, and the SDD is S-attributed

The underlying grammar is LL, and the SDD is L-attributed

我们可以使用 SDT 来实现两类重要的 SDD:

基础语法是 LR, SDD 是 S 属性的

基础语法是 LL, SDD 是 L 属性的

底层语法是 LR, 而 SDD 是 S 属性的:

- **LR 语法:** 这是一种可以用自底向上的方式进行解析的语法。LR 解析器从左到右读取输入, 但决策是基于右侧的上下文进行的。它使用一个堆栈来存储已经读取的符号, 并在适当的时候进行规约。

- **S 属性的 SDD:** 这意味着所有属性都是合成属性, 它们只从子节点传递给父节点, 没有从父节点传递给子节点的继承属性。这对于自底向上的 LR 解析是非常合适的, 因为我们从叶子节点开始构建并向上移动, 因此只需要合成属性即可。

底层语法是 LL, 而 SDD 是 L 属性的:

- **LL 语法:** 这是一种可以用自顶向下的方式进行解析的语法。LL 解析器从左到右读取输入, 并基于左侧的上下文来进行决策。它通常使用递归下降的方法, 直接从起始符号开始, 并尝试应用各种产生式。

- **L 属性的 SDD:** 这种 SDD 除了合成属性外, 还包括继承属性, 但这些继承属性只能依赖于它左边的兄弟节点或其父节点的属性。由于 LL 解析是自顶向下的, 所以可以在解析过程中计算这些继承属性。

Postfix Translation Schemes

- If the grammar of an SDD is LR, and the SDD is S-attributed, then we can construct a *postfix SDT* (后缀SDT) to implement the SDD in bottom-up parsing

- Semantic actions always appear at the end of productions (hence “postfix”)

PRODUCTION	SEMANTIC RULES			
1) $L \rightarrow E \mathbf{n}$	$L.val = E.val$ SDT	$L \rightarrow E \mathbf{n}$	$\{ \text{print}(E.val); \}$	SDT
2) $E \rightarrow E_1 + T$	$E.val = E_1.val + T.val$	$E \rightarrow E_1 + T$	$\{ E.val = E_1.val + T.val; \}$	
3) $E \rightarrow T$	$E.val = T.val$	$E \rightarrow T$	$\{ E.val = T.val; \}$	
4) $T \rightarrow T_1 * F$	$T.val = T_1.val \times F.val$	$T \rightarrow T_1 * F$	$\{ T.val = T_1.val \times F.val; \}$	
5) $T \rightarrow F$	$T.val = F.val$	$T \rightarrow F$	$\{ T.val = F.val; \}$	
6) $F \rightarrow (E)$	$F.val = E.val$	$F \rightarrow (E)$	$\{ F.val = E.val; \}$	
7) $F \rightarrow \mathbf{digit}$	$F.val = \mathbf{digit.lexval}$	$F \rightarrow \mathbf{digit}$	$\{ F.val = \mathbf{digit.lexval}; \}$	

This is possible because in bottom-up parsing, before reducing to a production head, the grammar symbols in the production body have been visited and their synthesized attributes have been computed.

如果 SDD 的语法是 LR，而且 SDD 是 S 属性的，那么我们就可以构造一个后缀 SDT（后缀 SDT），在自下而上的解析中实现 SDD 语义动作总是出现在制作的末尾（因此称为 “后缀”）。

之所以能做到这一点，是因为在自下而上的解析过程中，在还原到生产头之前，已经访问过生产体中的语法符号，并计算了它们的合成属性。

Parser-Stack Implementation of Postfix SDT's

- Postfix SDT's can be implemented during LR parsing by executing the actions when reductions occur
- The synthesized attributes can be placed along with the grammar symbols on the stack

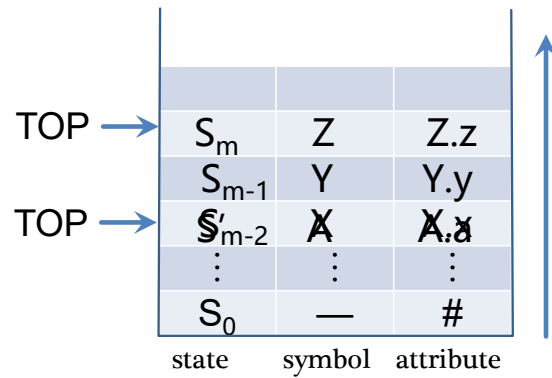
	X	Y	Z	State/grammar symbol
	$X.x$	$Y.y$	$Z.z$	Synthesized attribute(s)

↑
top

If we do reduction using $A \rightarrow XYZ$, then the attributes of A can be calculated based on the attributes of X , Y , and Z , which are already on the stack.

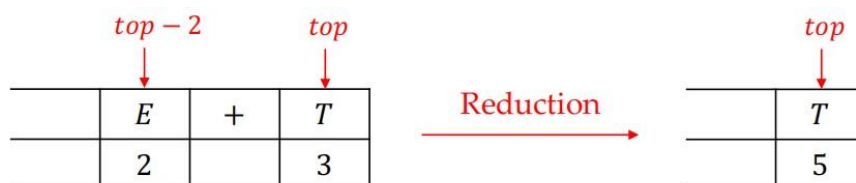
在 LR 解析过程中，可以通过在还原发生时执行以下操作来实现后缀 SDT 合成的属性可与语法符号一起放在堆栈中

如果我们使用 $A \rightarrow XYZ$ 进行还原，那么 A 的属性就可以根据堆栈中已经存在的 X 、 Y 和 Z 计算出来。就可以根据堆栈中已经存在的 X 、 Y 和 Z 的属性计算出 A 的属性。



The Calculator Example

PRODUCTION	ACTIONS
$L \rightarrow E \mathbf{n}$	{ $\text{print}(\text{stack}[\text{top} - 1].\text{val});$ $\text{top} = \text{top} - 1;$ }
$E \rightarrow E_1 + T$	{ $\text{stack}[\text{top} - 2].\text{val} = \text{stack}[\text{top} - 2].\text{val} + \text{stack}[\text{top}].\text{val};$ $\text{top} = \text{top} - 2;$ }
$E \rightarrow T$	
$T \rightarrow T_1 * F$	{ $\text{stack}[\text{top} - 2].\text{val} = \text{stack}[\text{top} - 2].\text{val} \times \text{stack}[\text{top}].\text{val};$ $\text{top} = \text{top} - 2;$ }
$T \rightarrow F$	
$F \rightarrow (E)$	{ $\text{stack}[\text{top} - 2].\text{val} = \text{stack}[\text{top} - 1].\text{val};$ $\text{top} = \text{top} - 2;$ }
$F \rightarrow \text{digit}$	

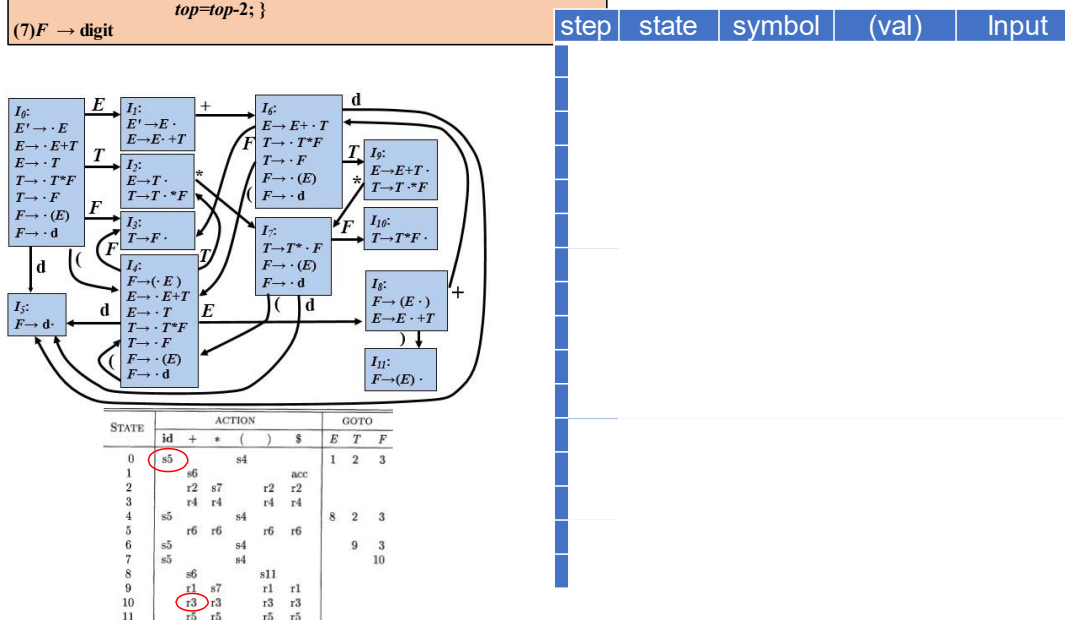


- 在许多语言中，保留字或关键词（如 then）也符合标识符模式

- 问题：搜索标识符的过渡图也能识别保留字

PRODUCTIONS	ACTIONS
(1) $E' \rightarrow E$	{ print (stack[top].val); }
(2) $E \rightarrow E_1 + T$	{ stack[top-2].val = stack[top-2].val + stack[top].val ; top=top-2; }
(3) $E \rightarrow T$	
(4) $T \rightarrow T_1 * F$	{ stack[top-2].val = stack[top-2].val $\times$ stack[top].val ; top=top-2; }
(5) $T \rightarrow F$	
(6) $F \rightarrow (E)$	{ stack[top-2].val = stack[top-1].val; top=top-2; }
(7) $F \rightarrow \text{digit}$	

$$\bullet 3*5+4n$$



Uses of SDT's

- We can use SDT's to implement two important classes of SDD's:

The underlying grammar is LR, and the SDD is S-attributed

The underlying grammar is LL, and the SDD is L-attributed

SDT's for L-Attributed SDD's

- L-attributed SDD's can be implemented during top-down parsing, if the underlying grammar is LL
- The way of turning an L-attributed SDD into an SDT is to place semantic actions at appropriate positions in the concerned production $A \rightarrow X_1X_2...X_n$
 - Embed the action that computes the **inherited attributes** for a nonterminal X_i immediately **before** X_i in the production body
 - Place the actions that compute a **synthesized attribute** for the production head **at the end** of the production body

如果基础语法是 LL，那么 L 归因 SDD 可以在自上而下的解析过程中实现。

将 L 属性 SDD 转化为 SDT 的方法是将语义动作放在相关语篇 $A \rightarrow X_1X_2...X_n$ 的适当位置上。

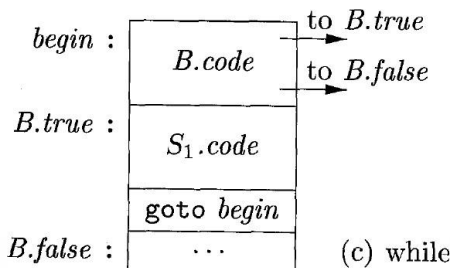
将计算非终端 X_i 的继承属性的动作紧接在生产体中 X_i 的前面

将为语篇头部计算合成属性的操作放在语篇主体的末尾

An L-Attributed SDD

- The SDD generates labels for the *while* loop

$S \rightarrow \text{while} (B) S_1$ $\text{begin} = \text{newlabel}();$
 $B.\text{true} = \text{newlabel}();$
 $B.\text{false} = S.\text{next};$
 $S_1.\text{next} = \text{begin};$
 $S.\text{code} = \text{label}(\text{begin}) \parallel B.\text{code} \parallel \text{label}(B.\text{true}) \parallel S_1.\text{code} \parallel \text{gen}('goto' \text{ begin});$



Inherited attributes: S.next, B.true, B.false

Synthesized attribute: S.code, S1.code

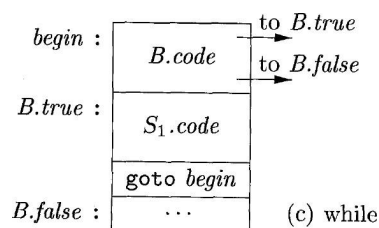
S.Begin while 语句开头

C.True S1 的第一条语句

Turning into an SDT

- Semantic actions:

- $\text{begin} = \text{newlabel}(); B.\text{true} = \text{newlabel}();$
- $B.\text{false} = S.\text{next};$
- $S_1.\text{next} = \text{begin};$
- $S.\text{code} = \dots;$



- According to the rules of action placement:

- b) should be placed before C, c) should be placed before S_1 , and d) should be placed at the end of the production body
- a) can be placed at the very beginning; there is no constraint

$S \rightarrow \text{while} (\{ \text{begin} = \text{newlabel}(); B.\text{true} = \text{newlabel}(); B.\text{false} = S.\text{next}; \}$
 $B) \quad \{ S_1.\text{next} = \text{begin}; \}$
 $S_1 \quad \{ S.\text{code} = \text{label}(\text{begin}) \parallel B.\text{code} \parallel \text{label}(B.\text{true}) \parallel S_1.\text{code} \parallel \text{gen}('goto' \text{ begin}); \}$

- 语义操作:

- a) `L1 = new(); L2 = new();`
- b) `C.false = S.next; C.true = L2;`
- c) `S1.next = L1;`
- d) `S.code = ... ;`

根据动作位置规则,

b) 应放在判断语句 C 之前, c) 应放在 S1 语句之前 (计算 S1 的继承属性), d) 应放在产生式末尾 (产生式左部 S 的综合属性计算应该在产生式右部所有文法符号的属性都计算完才能计算)

- a) 可以放在开头; 没有限制条件

Translation During Recursive-Descent Parsing

- Many translation applications can be addressed using L -attributed SDD's. It is possible to **extend a recursive -descent parser to implement L -attributed SDD's** as follows:
 - A recursive-decent parser has a function A for each nonterminal A
 - Use the arguments of function A to pass A's **inherited attributes**
 - Return the **synthesized attributes** of A when the function A completes
- With the above extension, in the body of the function A, we need to both **parse** and **handle attributes**

许多翻译应用都可以使用 L 归因 SDD 来解决。如下所示, 我们可以扩展递归-下降解析器来实现 L 属性 SDD:

递归-迭代解析器为每个非终端 A 都有一个函数 A

使用函数 A 的参数传递 A 的继承属性

当函数 A 完成时, 返回 A 的合成属性

通过上述扩展，在函数 A 的主体中，我们需要同时解析和处理属性

算术表达式计算的例子

➤ 例

SDT

1) $T \rightarrow F \{ T'.inh = F.val \} T'$
 $\{ T.val = T'.syn \}$

2) $T' \rightarrow *F \{ T_1'.inh = T'.inh \times F.val \} T_1'$
 $\{ T'.syn = T_1'.syn \}$

3) $T' \rightarrow \varepsilon \{ T'.syn = T'.inh \}$

4) $F \rightarrow \text{digit} \{ F.val = \text{digit.lexval} \}$

为每个非终结符A构造一个函数，A的每个继承属性对应该函数的一个形参，函数的返回值是A的综合属性值

对出现在A产生式右部中的每个文法符号的每个属性都设置一个局部变量

对于每个动作，将其代码复制到语法分析器，并把对属性的引用改为对相应变量的引用

```

T'syn T'(token, T'inh)
{
  D: Fval, T1'inh, T1'syn;
  if token="*" then
  {
    Getnext(token);
    Fval=F(token);
    T1'inh= T'inh×Fval;
    Getnext(token);
    T1'syn=T1'(token, T1'inh);
    T'syn=T1'syn;
    return T'syn;
  }
  else if token= "$" then
  {
    T'syn= T'inh;
    return T'syn;
  }
  else Error;
}

```

➤ 例

SDT

1) $T \rightarrow F \{ T'.inh = F.val \} T'$
 $\{ T.val = T'.syn \}$

2) $T' \rightarrow *F \{ T_1'.inh = T'.inh \times F.val \} T_1'$
 $\{ T'.syn = T_1'.syn \}$

3) $T' \rightarrow \varepsilon \{ T'.syn = T'.inh \}$

4) $F \rightarrow \text{digit} \{ F.val = \text{digit.lexval} \}$

Tval T(token)

```

{
  D: Fval, T'inh, T'syn;
  Fval = F(token);
  T'inh = Fval;
  Getnext(token);
  T'syn = T1'(token, T'inh);
  Tval = T'syn;
  return Tval;
}

```

➤ 例

SDT

- 1) $T \rightarrow F \{ T'.inh = F.val \} T'$
 $\{ T.val = T'.syn \}$
- 2) $T' \rightarrow *F \{ T_1'.inh = T'.inh \times F.val \} T_1'$
 $\{ T'.syn = T_1'.syn \}$
- 3) $T' \rightarrow \varepsilon \{ T'.syn = T'.inh \}$
- 4) $F \rightarrow \text{digit} \{ F.val = \text{digit.lexval} \}$

Fval F(token)

```
{  
    if token ≠ digit then Error ;  
    Fval=token.lexval ;  
    return Fval;  
}
```

➤ 例

SDT

- 1) $T \rightarrow F \{ T'.inh = F.val \} T'$
 $\{ T.val = T'.syn \}$
- 2) $T' \rightarrow *F \{ T_1'.inh = T'.inh \times F.val \} T_1'$
 $\{ T'.syn = T_1'.syn \}$
- 3) $T' \rightarrow \varepsilon \{ T'.syn = T'.inh \}$
- 4) $F \rightarrow \text{digit} \{ F.val = \text{digit.lexval} \}$

Desent()

```
{  
    D: Tval;  
    Getnext(token);  
    Tval = T(token);  
  
    if token ≠ "$" then Error ;  
    return ;  
}
```